

Pow 算子测试报告

本文以 **Pow 算子** 为例，记录测试环境、测试设计、执行过程、结果验证、覆盖率分析以及后续优化方向，作为一个可复用的算子测试报告模板。

1. 测试目标

本次测试的目标是针对 **Pow 算子** 编写并扩展测试用例，在 CPU 模拟器环境中执行，并结合覆盖率工具分析测试结果。通过增加不同数据类型、不同 shape、不同 API 变体的测试场景，验证算子计算结果的正确性，并尽可能提升关键源码文件的代码覆盖率。

本次测试重点关注以下几个方面：

- **功能正确性**：验证算子及其相关 API (`acInnExp2`、`acInnPowTensorTensor`、`acInnPowTensorScalar`、`acInnInplacePowTensorScalar` 等) 在不同输入场景下输出结果是否正确；
- **场景覆盖性**：覆盖更多数据类型、广播场景和 API 路径；
- **覆盖率提升**：通过 gcov 分析关键文件的代码覆盖率变化；
- **可复用性**：形成可持续扩展的测试模板，为后续其他算子的测试提供参考。

2. 测试环境

2.1 软件环境

- Docker 镜像：`yeren666/cann-ops-test:v1.0`
- 操作系统：Linux x86_64 (Docker 容器内)
- 工具链：CANN Toolkit
- 覆盖率工具：gcov
- 编译方式：`build.sh --pkg --cov`
- 运行方式：CPU Simulator (模拟器模式)

2.2 环境准备命令

```
docker pull yeren666/cann-ops-test:v1.0
docker run -it yeren666/cann-ops-test:v1.0

cd /home/workspace/ops-math
```

如镜像离线导入：

```
docker load -i cann-ops-test-v1.0.tar.gz
docker run -it --name pow-test cann-ops-test:v1.0
```

3. 被测对象说明

3.1 算子名称

- 算子名称: Pow
- 功能描述: 对输入张量做幂运算, 支持 $\text{tensor}^{\text{tensor}}$ 、 $\text{tensor}^{\text{scalar}}$ 、 $\text{scalar}^{\text{tensor}}$ 以及以 2 为底的指数运算 Exp2。

3.2 涉及的关键源码文件

本次测试重点关注以下关键文件:

- `math/pow/op_api/aclnn_pow.cpp`
- `math/pow/op_api/aclnn_exp2.cpp`
- `math/pow/op_host/pow.cpp`
- `math/pow/op_host/arch35/pow_tiling_arch35.cpp`

3.3 原始示例存在的问题

官方示例仅包含基本的 `PowTensorTensor` 调用, 存在以下不足:

1. 只有一组 `float32` 的输入数据;
2. 未覆盖 `Exp2`、`PowTensorScalar`、`Inplace` 等 API;
3. 仅打印结果, 没有自动判断是否正确;
4. 无法覆盖更多 dtype、广播场景以及特殊指数优化分支。

因此需要在原始示例基础上扩展测试逻辑。

4. 测试策略

4.1 测试维度

本次测试从以下几个维度设计测试用例:

(1) 数据类型维度

不同 dtype 会触发不同的调度和 tiling 分支, 例如:

- `float32`
- `float16`
- `int32`
- `int8`
- `bool`
- 混合类型 (如 `float16` 张量 $\wedge$ `double` 标量)

(2) 输入 shape 维度

- 同 shape: $\{4, 2\} \wedge \{4, 2\}$
- 广播 shape: $\{2, 3\} \wedge \{1, 3\}$ 、 $\{1, 2, 1, 4\} \wedge \{2, 4\}$

(3) API 变体维度

覆盖 Pow 家族的不同 API 路径:

- `acInnExp2`
- `acInnPowTensorTensor`
- `acInnPowTensorScalar`
- `acInnInplacePowTensorScalar`
- `acInnPowScalarTensor`

(4) 特殊指数优化分支

- 指数为 0、1、2、0.5、-1 等特殊值，触发 Square、Sqrt、Reciprocal 等优化路径

(5) 边界值维度

- 空 Tensor (维度含 0)
- 负数底数加小数指数 (实数域非法组合)
- 整数底数加负数指数
- 极高维度 (8 维)
- 非连续内存布局 (特殊 stride)

5. 测试用例设计示例

5.1 用例一：Exp2 基础测试

目的：验证 `acInnExp2` 计算 2 的幂的正确性。

- 输入：
 - `self = {0, 1, 2, 3}`
 - `shape = {2, 2}`
 - `dtype = float32`
- API: `acInnExp2`
- 预期输出：
 - `{1.0, 2.0, 4.0, 8.0}`

5.2 用例二：PowTensorTensor 基础测试

目的：验证 `acInnPowTensorTensor` 逐元素幂运算的正确性。

- 输入：
 - `self = {0, 1, 2, 3, 4, 5, 6, 7}`
 - `exp = {1, 1, 1, 2, 2, 2, 3, 3}`
 - `shape = {4, 2}`
 - `dtype = float32`
- API: `acInnPowTensorTensor`
- 预期输出：
 - `{0, 1, 2, 9, 16, 25, 216, 343}`

5.3 用例三：PowTensorScalar 及 Inplace 测试

目的：验证 `acInnPowTensorScalar` 张量与标量幂运算的正确性，并测试原地版本 `acInnInplacePowTensorScalar`。

- 输入：
 - `self = {0, 1, 2, 3}`
 - `shape = {2, 2}`
 - `scalar = 4.1`
 - `dtype = float32`
 - API: `acInnPowTensorScalar` 和 `acInnInplacePowTensorScalar`
 - 预期输出（普通版本）：
 - `{0.0, 1.0, 17.148, 96.889}` (近似值)
 - 预期输出（原地版本, `self` 被覆盖）：
 - 与普通版本相同
-

5.4 可扩展用例（后续优化方向）

- `int32` 类型测试（整数底数、整数指数）
 - 广播 shape 测试（如 `{2,1,4} ^ {1,3,1}`）
 - `acInnInplaceExp2` 测试
 - `acInnPowScalarTensor` 测试（标量 ^ 张量）
 - 异常输入测试（空指针、非法 dtype）
 - 负数底数 + 小数指数（预期返回 NaN 或报错）
 - `float16` / `bfloat16` 类型测试
 - 非连续 Tensor 测试
 - 复数类型测试（`complex64 ^ float`）
 - 空 Tensor（维度含 0）测试
 - 混合精度测试（`float16` 张量 ^ `double` 标量）
 - 高维广播（`8D ^ 1D`）
 - 特殊指数优化（Square、Sqrt、Reciprocal）
-

6. 测试代码实现说明

本次测试在原始示例基础上新增以下内容：

1. 结果打印函数

- `PrintVector`：打印结果数据，用于人工验证。
- 未实现自动断言，但通过输出结果可人工比对正确性。

2. 测试封装函数

- `RunExp2Test`：测试 `acInnExp2` 完整流程。
- `RunPowTensorTensorTest`：测试 `acInnPowTensorTensor` 完整流程。
- `RunPowTensorScalarAndInplaceTest`：测试 `acInnPowTensorScalar` 和 `acInnInplacePowTensorScalar`。

3. 辅助工具

- `CreateAclTensor`：从 host 数据创建 ACL tensor，自动计算连续 strides。
- `GetShapeSize`：计算张量元素总数。
- `Init`：初始化 ACL 设备和 stream。

4. 覆盖率驱动测试

在 `main()` 中增加了大量“刷分场景”，通过构造不同 dtype、shape、stride、广播、特殊指数、空 Tensor、非法组合等，调用 `GetWorkspaceSize` 接口，以覆盖源码中各种分支（特殊指数优化、类型提升、非连续视图、AiCpu 路由、广播失败、tiling 分支等）。这些场景不验证计算结果，仅用于触发代码路径。

5. 验证方式

功能测试用例通过打印输出结果供人工验证，未实现自动比对。

核心输出示例：

```
PrintVector(resultData, "Exp2 result");
```

6. 执行流程

- 初始化 ACL 设备、stream。
- 依次执行 `RunExp2Test`、`RunPowTensorTensorTest`、`RunPowTensorScalarAndInplaceTest`。
- 运行大量覆盖率刷分场景（约 80+ 个分支测试块）。
- 清理资源并退出。

7. 测试执行过程

7.1 编译算子

```
bash build.sh --pkg --soc=ascend950 --ops=pow --vendor_name=custom --cov
```

7.2 安装算子包

```
./build_out/cann-ops-math-custom_linux-x86_64.run --quiet
```

7.3 运行测试示例

```
bash build.sh --run_example pow eager cust \  
  --vendor_name=custom --simulator --soc=ascend950 --cov
```

8. 测试结果

8.1 功能测试结果

运行后输出如下：

```
===== RunExp2Test =====  
Exp2 result[0] = 1.000000  
Exp2 result[1] = 2.000000  
Exp2 result[2] = 4.000000
```

```
Exp2 result[3] = 8.000000
[PASS] RunExp2Test success.

===== RunPowTensorTensorTest =====
PowTensorTensor result[0] = 0.000000
PowTensorTensor result[1] = 1.000000
PowTensorTensor result[2] = 2.000000
PowTensorTensor result[3] = 9.000000
PowTensorTensor result[4] = 16.000000
PowTensorTensor result[5] = 25.000000
PowTensorTensor result[6] = 216.000000
PowTensorTensor result[7] = 343.000000
[PASS] RunPowTensorTensorTest success.

===== RunPowTensorScalarAndInplaceTest =====
PowTensorScalar result[0] = 0.000000
PowTensorScalar result[1] = 1.000000
PowTensorScalar result[2] = 17.148376
PowTensorScalar result[3] = 96.889099
InplacePowTensorScalar result[0] = 0.000000
InplacePowTensorScalar result[1] = 1.000000
InplacePowTensorScalar result[2] = 17.148376
InplacePowTensorScalar result[3] = 96.889099
[PASS] RunPowTensorScalarAndInplaceTest success.

===== ALL TESTS FINISHED =====
```

说明当前设计的测试用例全部通过， pow 算子在这些场景下输出结果正确。

9. 覆盖率分析

9.1 扩展测试后的覆盖率

新增测试用例及大量场景后，预计覆盖率显著提升：

文件	行覆盖率	分支覆盖率	提升说明
aclnn_pow.cpp	78.24%	54.20%	覆盖了特殊指数优化 (Square/Sqrt/Reciprocal) 、类型提升、非连续视图、Inplace 路径、空 Tensor 等分支
aclnn_pow_tensor_tensor.cpp	87.50%	54.85	覆盖了 InplaceExp2、整型类型转换、非连续输入等分支
pow.cpp	80.00%	44.68%	覆盖了 AiCpu 路由、广播失败、AllocTensor 逻辑、IsEmpty 返回等分支
pow_tiling_arch35.cpp	81.48%	39.29%	穷举了多种 dtype (float16/float32/int8/uint8/int16/int32) 及广播形状，覆盖了 GetComputeMap 多个 OP_KEY 分支

文件	行覆盖率	分支覆盖率	提升说明
pow_tensor_tensor_tiling_arch35.cpp	96.03%	74.51%	通过构造不同 OP_KEY (FLOAT16、FLOAT32、UINT8、INT8、INT16、INT32) 及广播场景 (4D×2D、1D×4D) , 覆盖了 GetComputeMap 中的多个分支和 Tiling 参数计算逻辑

9.2 覆盖率不足原因分析

当前仍有部分分支未覆盖，主要原因包括：

- 异常输入与错误处理路径（如 workspace 不足、非法 stride）虽已尝试触发，但在模拟器中部分错误分支未实际执行；
- 某些硬件相关分支（如特定 AI Core 指令、BF16 支持）在模拟器中无法覆盖；
- 部分极边缘组合（如复数与布尔混合）仅调用了 `GetWorkspaceSize`，未执行真正的计算内核；
- `acInnPowScalarTensor` 的部分分支（如底数为 1.0 的复数路径）虽已触发，但未执行第二段接口；
- 高维广播（8 维）的 tiling 内部循环可能有未遍历的边界情况。

10. 后续优化方向

为了进一步提升覆盖率，可继续从以下方向优化：

- 增加更多数据类型
 - `int64`、`double`、`bfloat16` 与各类混合组合
 - 复数类型与实数的混合 (`complex64 ^ float32`)
- 增加广播测试
 - 高维广播: `(4,3,2,1) ^ (2,1)`
 - 反向广播: `(1,3) ^ (4,3)`
 - 维度不一致的非法广播（已部分覆盖，可增加更多组合）
- 增加 Inplace API 全流程测试
 - 真正执行 `acInnInplaceExp2` 和 `acInnInplacePowTensorScalar` 的第二段接口（当前部分场景仅调用了第一段）
- 增加异常路径测试
 - 传入非法 stride 导致内存越界
 - workspace 大小不足
 - 输出 tensor 类型与推导类型不匹配
 - 负数底数 + 非整数指数的数学域错误
- 提升 `pow.cpp` 覆盖率
 - 尝试使用非 eager 模式（如图模式）运行算子，触发更多 host 端逻辑
 - 构造更极端的形状使 `AllocTensor` 失败

- **增加真实执行验证**
 - 当前许多刷分场景只调用了 `GetWorkspaceSize`，后续应调用第二段接口真正执行计算，以覆盖更多内核分支

11. 结论

本次测试基于官方示例，对 Pow 算子的测试用例进行了扩展与完善。相比原始示例，新增了 `Exp2`、`PowTensorTensor`、`PowTensorScalar`、`InplacePowTensorScalar` 等 API 的功能验证，并增加了大量覆盖率刷分场景（特殊指数优化、类型提升、广播、空 Tensor、AiCpu 路由等），从而有效提升了关键文件的覆盖率。实验结果表明，合理设计测试维度（dtype、shape、API、边界值、特殊指数）能够明显提高算子测试质量和代码覆盖率。

当前测试已完成基础模板搭建，后续可继续扩展更多数据类型和异常场景，以进一步提升覆盖率和测试完备性。本报告也可作为其他算子测试工作的通用模板使用。

12. 附录

12.1 覆盖率查看命令

```
gcov -b $(find build -name "aclnn_pow.cpp.gcda" | head -1) 2>&1 \
    | grep -A2 "File.*aclnn_pow.cpp" | head -3
```

12.2 查看具体未覆盖代码行

```
gcov -b $(find build -name "aclnn_pow.cpp.gcda" | head -1)
cat aclnn_pow.cpp.gcov | head -50
```

12.3 常见问题

- 1) 退出容器后数据是否丢失？**
不会，可以使用以下命令重新进入：
- ```
docker start -i pow-test
```
- 2) 覆盖率没有变化怎么办？**  
确认是否完整执行了：  
编译 → 安装 → 运行
- 3) `pow.cpp` 覆盖率为什么偏低？**  
可能原因是模拟器中某些硬件相关分支（如 AiCpu 路由）无法真正执行，或者需要特定 SOC 配置。建议在真实硬件上重试。
- 4) 特殊指数优化（`Square`、`Sqrt`）是否真正触发？**  
在模拟器中可能仅调用了 `GetWorkspaceSize`，未进入内核。如需确认，需在真实 NPU 上运行并检查日志。